

공부 노트

실험 설계 정리 + HamNet · 2026-09-14 수정판
전문

공부용

1부 실험 설계 이해 정리 (16절)

2부 HamNet 모델 공부 (15절)

노선에 올린 수정판과 같은 내용

곽동신 · 2026-09-14

블루베리 세그멘테이션 연구 · 안홍렬 교수님 연구실

1부. 실험 설계 이해 정리

실험 설계 이해 정리 및 외부 AI 교차검토 (2026-09-14 수정판)

작성자: 곽동신 작성: 2026-09-12 / 최종 수정: 2026-09-14

목적: 9월 15일(화) 랩미팅에서 교수님의 파일럿 → 선별 → 본실험 설계를 순서도로 설명하기 위한 정리 기준: 교수님 결정 29·30·31, 9월 14일 14시까지 확인한 실험 상태, 그리고 2026-09-14 AI 교차 검토(1차 원문 검토: 서버

Claude·Codex·Gemini·DeepSeek / 2차 수정판 교정: Codex·Gemini. Grok 은 학교 게이트웨이 시간초과로 답 없음)

핵심 요약

- 전체 후보는 헤더 12종 × 백본 12종 = 144조합이다.
- 데이터의 20%를 떼어 둔 파일럿에서 144조합을 비교해 후보 집합 C(12~23조합)를 고른다.
- 나머지 80% 본실험에서는 144조합 전체를 M1 에서 한 번 평가하고, 후보 C 만 M2~M5 에서 추가 평가한다.
- 9월 11일 결정 29·30 으로 파일럿 분할이 3-fold 에서 4분할·4회전으로 바뀌었고 파일럿0(64런)이 신설됐다.
- 9월 13일 결정 31 로 파일럿0 을 구간별 체크포인트를 저장하며 전부 다시 실행했다. 파일럿0 숫자는 runs_v2 것만 인용한다.
- 9월 14일 14시 기준 파일럿0 v2 는 48/64 완주(포도·블루베리·사과 각 16, 복숭아 진행 중)다.

1. 이 정리를 한 이유

- 9월 8일 랩미팅에서 교수님이 설계한 파일럿 → 후보 선별 → 본실험 구조를 이해하고, 9월 15일 랩미팅에서 순서도로 설명하는 과제를 받았다.
- 자료를 준비하던 중 9월 11일에 설계가 바뀐 것을 확인해 순서도 1판을 폐기하고 다시 만들었다.
- 내 이해가 맞는지 보려고 같은 원문과 과제를 GPT, 로컬 Claude, DeepSeek, Gemini 에 주고 서버 Claude 정리와 대조했다(9월 12일).
- 9월 13일 결정 31 로 파일럿0 실행 방식이 다시 바뀌어

순서도·대본·런 수를 갱신했다.

14일에 고친 곳

9월 14일에는 이 문서와 HamNet 문서를 서버 Claude·Codex·Gemini·DeepSeek 에게 검토시키고, 고친 수정판을 다시 Codex·Gemini 에게 교정시켜 아래 내용을 고쳤다(고친 곳은 각 절에 «14일 수정» 으로 표시). Grok 도 시도했으나 학교 게이트웨이의 120초 제한으로 답을 받지 못했다.

2. 전체 실험 설계

한 문장 설명: 144개 모델 조합을 데이터 20%에서 먼저 선별한 뒤, 나머지 80%에서 전체 조합의 1차 성능과 선별 조합의 반복 성능을 평가하는 2단계 벤치마크다.

단계별 역할

- C1 — 변경 전 파일럿 이력 보존. 48런, 3-fold, 300 epoch 완주. 현행 설계와 섞지 않고 이력으로만 보존.
- 파일럿0 — 학습 epoch 예산 결정. 대표 2×2 조합 $\times$ 4데이터셋 $\times$ 4회전 = 64런, 800 epoch.
- 선별 P — 후보 조합 C 선정. $144\text{조합} \times 4\text{데이터셋} \times 4\text{회전} = 2,304\text{런}$.
- 본실험 M1 — 전체 조합의 1차 비교. 나머지 80%에서 144조합 전체 평가. $144 \times 4 = 576\text{런}$.
- 본실험 M2~M5 — 선별 조합의 반복 평가. 후보 C 만 추가 반복. $16 \times |C|(\text{후보 수 } 12\sim 23) = 192\sim 368\text{런}$.

런 수 기준

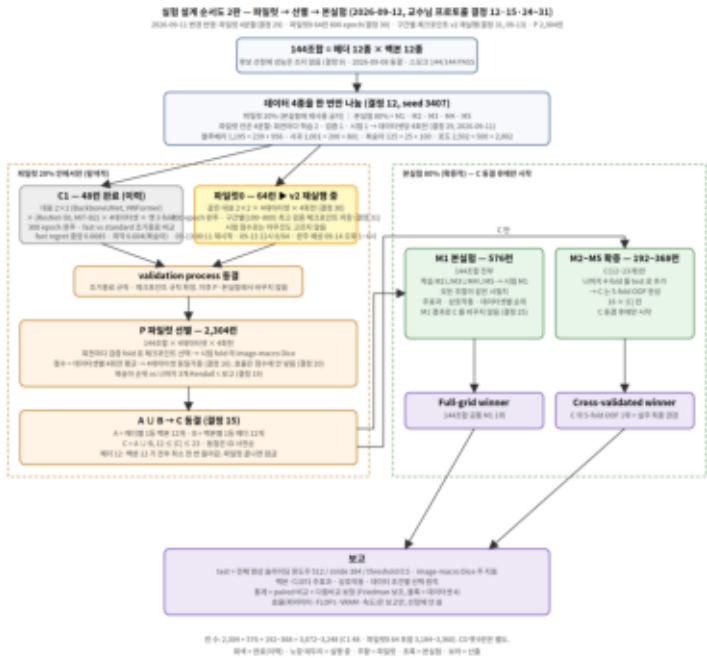
- 벤치마크 본체는 약 3,072~3,248런이다($P\ 2,304 + M1\ 576 + M2\sim M5\ 192\sim 368$).
- 보존 이력인 C1 48런과 파일럿0 v2 64런까지 더한 전체 집계는 약 3,184~3,360런이다(첫 파일럿0 실행 32런은 제외).
- 후보 C 의 최종 크기(12~23)에 따라 범위가 달라진다.

14일에 고친 곳

후보 C 를 고르는 규칙 «14일 수정 — 정의 추가»

- A = 헤더 12종 각각에 대해 가장 좋은 백본 1개씩(12조합).
- B = 백본 12종 각각에 대해 가장 좋은 헤더 1개씩(12조합).
- C = A $\cup$ B. 두 축의 12종을 모두 담고, 전체 최고 조합은 A와 B에 모두 들어가므로 최대 23개, 최소 12개다(결정 15). 동점은 ID 사전순으로 처리한다(결정 16).
- 선별 점수는 Dice 하나. 데이터셋별 4회전의 시험 fold Dice 평균을 4데이터셋 동일 가중으로 평균한다(결정 16). 체크포인트는 각 회전의 검증 fold 로 고른다.

순서도 2판



3. 데이터 분할

데이터셋별 장수 (전체 / 파일럿 20% / 파일럿 회전당
학습·검증·시험 / 본실험 80% / M1~M5 한 조각의 대략 크기
— 정확한 장수는 분할 정보 기준)

- 블루베리: 1,195 / 239 / 119~120 · 59~60 · 59~60 / 956 / 191
- 사과: 1,001 / 200 / 82~118 · 41~66 · 41~66 / 801 / 160
- 복숭아: 125 / 25 / 12~13 · 6~7 · 6~7 / 100 / 20
- 포도: 2,502 / 500 / 250 · 125 · 125 / 2,002 / 400

규칙

- 파일럿 20%는 본실험에 다시 사용하지 않는다.
- 같은 나무·영상·연속 프레임은 같은 분할에 둔다.
블루베리는 Camera N Video (M), 사과는 촬영 시퀀스를 그룹으로 사용한다.
- 포도와 복숭아는 그룹 정보가 없어 이미지 단위로 나눈다.
- 옛 실험 C0 에서 validation 으로 사용한 복숭아 23장과 포도 250장은 파일럿에 넣고
`prior_validation_exposure=true` 로 기록한다(그룹 정보가 없는 두 데이터셋이라 장 단위로 배정). 본실험에는 포함하지 않는다.
- 분할 정보는
`platform/04_experiments/260908_파일럿/protocol_splits/` 이다.
- 시험 fold 를 t, 검증 fold 를 t+1 로 순환시키고 나머지 2개 fold 로 학습한다. 모든 fold 가 시험과 검증에 각각 한 번씩 쓰인다.
- 복숭아는 회전당 학습 12~13장, 검증·시험 6~7장으로 매우 작다. 가중치는 바꾸지 않고 복숭아 순위와 나머지 세 데이터셋 순위의 Kendall's τ 를 별도로 보고한다.

4. 9월 11일 변경 사항 — 교수님 결정 29·30

파일럿 분할 방식 변경

- 파일럿 20% 구간을 3-fold 에서 4분할로 변경했다.
- 한 회전에서 학습 fold 2개, 검증 fold 1개, 시험 fold 1개를 사용한다.

- 데이터셋마다 시험 fold 를 바꾸며 총 4회전한다.
- 검증 fold 는 체크포인트 선택에 쓰고, 시험 fold 는 그 회전의 점수 측정에만 쓴다.
- 핵심은 «체크포인트를 고르는 데이터»와 «점수를 재는 데이터»를 분리하는 것이다. 대신 한 회전에서 실제 학습에 쓰는 비율은 파일럿의 2/3 에서 2/4 로 줄었다.

파일럿0 신설

- 대표 헤더 2종(BackboneUNet, VWFormer) × 대표 백본 2종(ResNet-50, MiT-B2) × 데이터셋 4종 × 4회전 = 64런.
- 각 런은 800 epoch 을 조기종료 없이 완주한다.
- 목적은 본 파일럿과 본실험에 적용할 epoch 예산을 먼저 정하는 것이다(결정 31 에서 명시).

런 수 변경

- 선별 P: 1,728런 → 2,304런.
- C1 48런은 완주했으며 변경 전 이력으로 보존한다.
- 9월 8일 순서도 1판은 변경 전 설계이므로 사용하지 않는다.

5. 9월 13일 변경 사항 — 교수님 결정 31

- 9월 13일 00시 11분, 교수님이 파일럿0 64런을 전부 다시 시작했다. GPU 8장을 모두 사용한다.
- 첫 실행은 런마다 best 와 final 체크포인트만 저장해서 «100, 200, ..., 800 epoch 중 어느 예산이 충분한가»를 비교할 수 없었다.
- 새 실행(파일럿0 v2)은 100·200·...·800 epoch 경계마다 그 시점까지의 최고 검증 모델을 저장한다. 런당 budget checkpoint 8개 + best + final.
- 각 checkpoint 를 전체영상 슬라이딩 방식으로 한 번씩 평가해 epoch 예산을 결정한다. 이 평가는 아직 하지 않았다.
- 첫 실행 결과 32런은 runs 폴더에 보존하되 분석과 발표에는 쓰지 않는다. 새 결과의 정본은 runs_v2 다.

6. 파일럿0 v2 중간 현황 «14일 수정 — 8/64 → 48/64

집계일: 2026-09-14 14:00 / 진행률: 48/64 완주(포도 16, 블루베리 16, 사과 16, 복숭아 진행 중) / 최종 결론 아님

무엇을 봤나

- 런마다 «epoch 예산 100 까지의 최고 검증 모델»과 «800 까지의 최고 검증 모델»의 센터크롭 시험 Dice 차이를 계산했다. 이득 = 800 예산 점수 - 100 예산 점수.

데이터셋별 결과 (100→800 이득 중앙값 / 최대, 400→800 이득 중앙값, best epoch 중앙값)

- 포도: +0.003 / +0.018, 0.000, 418
- 블루베리: +0.012 / +0.060, 0.000, 470
- 사과: +0.004 / +0.241, 0.000, 196
- 복숭아: 진행 중

해석

- 포도는 100 epoch 이면 거의 다 올라온다. 블루베리는 조금 더 걸린다. 400→800 이득의 중앙값은 세 데이터셋 모두 0.000 이다(개별 런의 최대는 +0.011).
- 옛 문장 «100 epoch 에서 거의 포화»는 포도 8런만 본 결과였다. 지금은 «데이터셋마다 다르고, 중앙값 기준으로 400 이후엔 거의 늘지 않는다»가 맞는 표현이다. 중앙값이므로 개별 런의 후기 개선까지 없다고 일반화하지 않는다.
- 사과의 최대 이득 +0.241 은 한 런(VWFormer × MiT-B2, 시험 fold 1)의 값이다. 이 런은 100 예산 시험 Dice 0.052, 800 예산 0.293 으로 둘 다 낮다. 최댓값 하나로 전체를 말하지 않는다.
- 사과는 시험 fold 1 회전에서 네 조합 모두 시험 Dice 가 0.05~0.63 인데 같은 런의 검증 Dice 는 0.83~0.85 다. 검증은 높고 시험만 낮으므로 그 시험 조각의 분포 차이가 의심되지만, 원인은 확인 중이다. 블루베리도 fold 1 이 가장 낮다(0.83~0.87, 나머지 0.92~0.95). 그룹 단위 분할의 영향일 수 있다.
- 이 값은 센터크롭 기준 중간 집계 결과다. 최종 판정은 budget checkpoint 별 전체영상 슬라이딩 평가로 한다.

원자료:

platform/04_experiments/260911_파일럿0/runs_v2/*/budget (center_crop_test_fg_dice), pilot0_summary.json

7. C1 에서 확인한 조기종료 특성

- C1 은 대표 2×2 조합을 4개 데이터셋과 옛 3-fold 에서 300 epoch 까지 학습한 48런이다. 현행 4분할 설계와 직접 비교하지 않고 조기종료 규칙을 검토한 이력으로 보존한다.
- standard: 80 epoch 이후 40 epoch 동안 개선이 없으면 종료. fast: 40 epoch 이후 10 epoch 동안 개선이 없으면 종료.
- fast 가 standard 보다 놓친 IoU 의 중앙값은 0.0085, 평균은 0.032 였다.
- 최악 사례는 복숭아 VWFormer $\times$ ResNet-50 fold2 로, fast 는 42 epoch 에 멈췄지만 standard 는 175 epoch 에서 개선되어 regret 이 0.604 였다.
- 데이터셋별 regret 중앙값은 포도 0.003, 사과 0.007, 블루베리 0.017, 복숭아 0.030 이었다.
- 평균 종료 시점은 standard 164 epoch, fast 63 epoch 였다.
- 즉 fast 규칙은 대부분 계산량을 줄였지만 저표본 데이터에서 큰 후기 개선을 놓칠 수 있었다. 이 결과만으로 규칙을 확정하지 않고 800 epoch 파일럿0 을 추가했다.

8. 비교 모델 12×12 구성

헤더(디코더) 12종 — ID, 모델, 연도·게재처, 계열

- H001 FCN — 2015 CVPR — CNN 단순 업샘플
- H002 U-Net-style decoder — 2015 MICCAI — 대칭 encoder-decoder (BackboneUNet 으로 구현)
- H003 Semantic FPN — 2019 CVPR — 피라미드 융합
- H004 PSPNet head — 2017 CVPR — pooling 문맥
- H005 DeepLabV3+ head — 2018 ECCV — atrous 문맥
- H006 UPerNet — 2018 ECCV — PPM+FPN 복합
- H007 OCRNet — 2020 ECCV — attention 문맥
- H008 AllMLPNet — 2021 NeurIPS — MLP 융합
- H009 FaPN — 2021 ICCV — deformable 정렬
- H010 U-MixFormer — 2025 WACV — cross-attention
- H011 VWFormer — 2024 ICLR — varying-window attention

- H025 HamNet (LightHam) — 2021 ICLR — 행렬분해 문맥
- 계열 분포: CNN·pooling 7종, attention 3종, MLP 1종, 행렬분해 1종. 연도 분포: 2015년 2종, 2017~2019년 4종, 2020~2021년 4종, 2024~2025년 2종.

백본(인코더) 12종 — ID, 모델, 연도·게재처, 계열

- B001 ResNet-50 — 2016 CVPR — residual CNN
- B002 ConvNeXt-T — 2022 CVPR — 현대 CNN (공식 ImageNet-1K 가중치)
- B004 HRNetV2-W32 — 2019 CVPR / 2020 TPAMI — 고해상도 병렬 CNN (projection 포함 30.882M)
- B005 Swin-T — 2021 ICCV — window Transformer
- B006 MiT-B2 — 2021 NeurIPS — 계층형 Transformer (SegFormer 인코더, 파일럿0 대표 백본)
- B007 UniFormer-S† — 2022 ICLR / 2023 TPAMI — CNN-Transformer hybrid (head_dim 64, 23.521M)
- B008 VMamba-T — 2024 NeurIPS — visual SSM (v2 s1l8)
- B016 MSCAN-B — 2022 NeurIPS — convolutional attention CNN
- B017 InternImage-T — 2023 CVPR — deformable CNN (DCNv3)
- B018 TransNeXt-Tiny — 2024 CVPR — aggregated-attention Transformer
- B019 MogaNet-S — 2024 ICLR — multi-order gated CNN
- B020 vHeat-T — 2025, 게재처 미기재 — PDE/spectral operator (라이선스 unspecified)
- 계열 분포: CNN 6종, Transformer 3종, Hybrid 1종, SSM 1종, PDE/operator 1종. 파라미터: backbone-only 실측 23.508M~30.882M.

동결 및 검증 상태

- 2026년 9월 8일 교수님이 백본 12종의 identity와 checkpoint를 확정했다.
- 144조합의 512×512 AMP smoke test는 144/144

통과했다.

- 근거:
platform/planning/frozen/260908_체크포인트_동결기록.md,
platform/planning/260905_02_모델_선정.md § 4· § 8

9. 최종 시험·평가 규칙

전체영상 추론

- 최종 test 는 원본 영상 전체를 512×512 window 로 나눠 추론한다.
- stride 는 384 로 고정해 window 가 25% 겹친다.
- threshold 는 0.5. TTA, 후처리, test-time resizing 은 쓰지 않는다.
- window 가 겹치는 영역은 단순평균으로 합치되, probability 평균과 logit 평균 중 무엇을 쓸지는 동결 전에 확정해야 한다.
- 센터크롭은 빠르지만 과실의 75~77%를 평가하지 못하고 데이터셋별 포착률도 다르다. 학습 중 검증, 파일럿0 의 예비 시험 평가, 512×512 효율 측정에 쓰고, 최종 정확도는 전체영상 슬라이딩으로 잰다.

14일에 고친 곳

체크포인트와 모든 프로토콜을 고정한 뒤 본실험의 test 를 한 번만 연다. «14일 수정» 파일럿0 의 구간별 체크포인트 평가는 예산을 정하는 개발 과정이므로 본실험 최종 시험과 구분한다.

지표와 통계

- 주 지표: image-macro Dice
- 보조 지표: IoU, Precision, Recall, Boundary IoU 또는 Boundary F-score, 객체 수준 지표
- 빈 전경 규칙: GT 와 예측이 모두 비면 Dice 1, 한쪽만 비면 Dice 0
- 전체영상 4,823장에는 빈 전경 영상이 0장이지만 학습 crop 에서는 생길 수 있다.
- 주 분석: paired 비교와 다중비교 보정. Friedman 검정은 보조·탐색적 분석.
- 독립 블록은 fold 가 아니라 데이터셋 4개.
- 저표본 복숭아: 가중치를 바꾸지 않고 Kendall's τ 와

limitation 을 보고.

- 효율성: 파라미터, FLOPs, VRAM, 512 입력 속도, 전체영상 처리시간을 성능과 분리해 보고.

10. 이 설계가 답하려는 연구 질문

교수님 260905_02_모델_선정.md § 5 의 취지는 다음 네 가지다.

- 어떤 헤더·백본 조합을 실무 최종 모델로 추천할 것인가.
- 좋은 백본은 디코더가 바뀌어도 계속 좋은가(구성요소의 주효과).
- 성능은 좋은 부품의 단순 합인가, 특정 헤더와 백본의 궁합(상호작용)이 중요한가.
- 최신 기법의 이득이 작고·밀집되고·겹치며·저표본인 과실 데이터에서도 재현되는가.

11. 아직 동결되지 않은 항목

- weight decay 와 warm-up 길이
- 데이터셋 크기 차이를 고려한 최대 update 수
- 학습 seed 의 개수와 값, deterministic 설정
- pretrained backbone 과 새 head 의 learning-rate multiplier
- batch normalization 처리 방식
- gradient clipping
- NaN·OOM 등 실패 판정과 재시작 횟수
- 슬라이딩 중첩부의 probability 평균 또는 logit 평균
- inference batch size 와 효율 측정 반복 횟수
- 매우 작은 객체의 크기 하한
- 문헌 검색 cutoff 날짜
- StrawDI 편입 여부
- 파일럿 이후 단계의 GPU 시간과 저장량
- 파일럿 P 가 시작되기 전에 위 설정과 validation process 를 동결해야 한다.

12. 논문에서 피해야 할 표현

- 후보 선정에 본실험 성능을 사용했다고 쓰지 않는다.
- 후보 C 를 줄인 이유를 통계 검출력이라고 쓰지 않는다. 이유는 계산량, 두 축의 대표성, 설명 단순성이다.
- M1 의 144조합 순위와 C 의 5-fold 순위를 같은 결과처럼 쓰지 않는다.
- 전체 데이터를 5-fold 교차검증했다고 쓰지 않는다. 정확한 표현은 «20% exclusive pilot, 144조합의 공통 M1 holdout, pilot-selected C 의 five-fold OOF» 다.
- 검색식과 screening flow 를 기록하지 않은 상태에서 전수조사라고 쓰지 않는다. candidate audit 또는 scoping review 로 표현한다.

13. 외부 AI 교차검토 결과 (9월 12일, 4개)

같은 자료와 과제를 GPT, 로컬 Claude, DeepSeek, Gemini 에 입력한 뒤 서버 Claude 결과와 대조했다.

공통적으로 맞게 이해한 내용

- 파일럿 20%와 본실험 80%의 분리
- AUB 선별 규칙

14일에 고친 곳

두 종류의 승자 구분 — Full-grid winner 는 144조합의 공통 M1 holdout 1위, Cross-validated winner 는 후보 C 의 본실험 5-fold(M1~M5) OOF 1위이며 실무 최종 권장은 후자다(파일럿 문서 § 5). «14일 수정 — 정의 보강»

14일에 고친 곳

본실험 데이터로 후보를 고르지 않는다는 원칙 «14일 수정 — 옛 문장 «시험 fold 를 모델 선택에 사용하면 안 된다»는 부정확했다. 파일럿 안의 시험 fold 점수는 후보 C 를 고르는 데 쓴다(결정 16). 쓰면 안 되는 것은 본실험 80% 데이터로 후보를 고르는 것이다.»

발견된 문제

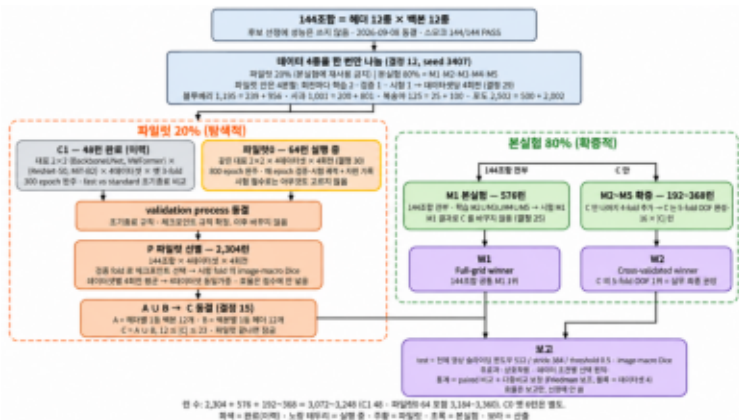
- Gemini 의 런 수 계산 오류 — «144개 전부를 5-fold 로 평가하면 약 3,600런»이라고 했지만 $144 \times 4\text{데이터셋} \times 5\text{-fold}$ 는 2,880런이다.

- 파일럿0 역할 과잉 해석 — Gemini 대본은 파일럿0 으로 모든 규칙을 동결한다고 단정했다. 9월 13일 결정 31 로 epoch 예산 판정 역할이 명확해졌다.
- 연구 질문 4개 생성 오류 — 네 AI 모두 자료만으로 연구 질문을 새로 만들었다. 입력에 교수님 02 문서가 빠져 있었기 때문이다. 실제 연구 질문은 02 문서 § 5 를 따른다.
- 원문 간 불일치 발견 — 로컬 Claude 가 교수님 문서 사이의 충돌을 찾았다. § 15 로 넘겼다.

최종 자료에 반영한 방식

- 전체 구조: 서버 Claude 순서도 2판 / 확인 질문: 로컬 Claude / 변경되지 않은 내용: GPT / 미정 항목 정리: DeepSeek / 런 수 소계: Gemini 결과를 재계산해 사용
- 비교 전문:
kds0206/문서/260912_external_ai_task/05_compare.md

GPT 가 그린 순서도(2026-09-12)



14. 9월 13~14일 자체 감사 결과

9월 13일 (서버 Claude 자체 감사, 확정 지적 27건 반영)

- 요일 오류 수정. 9월 11일은 금요일, 9월 15일 랩미팅은 화요일.
- O-1 슬라이딩 윈도우 추론 코드는 «없음»이 아니라 있음(260911_파일럿0/코드/semseg/eval/sliding.py). 서버 정본 03_code 에는 아직 미반영.
- 마무리 런 수 합계 문장과 근거 문서의 판 날짜 수정.

14일에 고친 곳

9월 14일 (1차: 서버 Claude·Codex·Gemini·DeepSeek 원문 검토 / 2차: Codex·Gemini 수정판 교정) «14일 추가»

- 1차 검토에서 네 AI 모두 문서 내용에서 사실 오류를 찾지 못했다. 교수님 원문과 대조한 항목: stride 384/25%, 복숭아 23장·포도 250장 prior exposure, 4,823장 빈 전경 0, C 12~23, 결정 29·30·31, 런 수, 발표자료 PDF 실제 20쪽.
- 고친 것: § 6 진행률과 «포화» 문장, § 2 AUB 정의 추가, § 13 «시험 fold 선택 금지» 표현, § 9 파일럿0 평가와 본실험 시험의 구분, § 15 확인 질문 축소, HamNet 문서 § 13 구현 확인 완료.
- 2차 교정에서 채택한 것: Codex 12건 중 11건(런 수 «신규» 표현, 동점 규칙, 한 조각 «대략», 복숭아·포도 장 단위 배정, 중앙값 일반화 완화, 사과 원인 «확인 중», «중간값»→«중간 집계», 센터크롭 용도, 이미지 블록 안내, NMF 입력 ReLU, 파일명 코드 표기) / Gemini 15건 중 8건(CD 설명, 수식 코드 표기, 빵 이름 대응, window 겹침, NMF2D 표기, |C| 병기 등).
- 채택하지 않은 외부 AI 의견: «사과·블루베리에서 최대 0.24 향상»을 대표 문장으로(한 런의 극단값) / «사과는 학습이 무너졌다, 학습률·증강 문제»(검증 Dice 정상) / «C는 regret 0.0085 기준으로 선별»(지어낸 규칙) / Gemini 의 «본실험 5-fold 를 4-fold 로 고쳐라»(본실험 80%는 M1~M5 5-fold 가 맞고 4분할은 파일럿 안 이야기) / «발표자료 PDF 파일명을 20p 로 바꿔라»(이미 전달한 파일이라 그대로 둬).

15. 교수님께 확인할 사항 «14일 수정 — 4개 → 2개 + 갱

발표 마지막에 여쭙볼 것

- VMamba-T 상태 — 안내.md 표에는 미결로 남아 있지만 00 문서 § 4 에서는 해소(v2 s1l8 동결)로 적혀 있다. 해소된 것으로 이해하고 있다.
- 선별 P 의 점수 산출 fold — 00 문서 § 1 은 «파일럿 validation Dice», 결정 16 은 시험 fold 를 가리킨다. 시험 fold 점수로 이해하고 있다.

옛 문서가 갱신되지 않은 것으로 보이는 곳(가볍게 알려드릴 것)

- 복숭아 파일럿의 회전당 학습 이미지 수 — 06 문서 § 4 는 약 17장, 01 문서 § 3 은 12~13장. 4분할 기준 12~13장이 맞다.
- 파일럿0 의 결정 범위 — 파일럿 문서 § 1 표에는 «미정»이지만 결정 31 이 epoch 예산 판정을 명시한다.

16. 작성한 자료

- 발표자료 PDF:
kds0206/문서/260912_final_deliverables/01_presentation_ (파일명과 달리 실제 20쪽)
- 쉬운 말 설명 11절:
kds0206/문서/260912_교수님_실험설계_쉬운말정리.md
- 순서도 2판·5분 대본·예상 질문:
kds0206/문서/260912_실험설계_순서도_2판.md (같은 이름의 .svg, .png)
- 12×12 모델 연도·계열표:
kds0206/문서/260912_노선불여넣기_12x12표_연도계열.md
- 외부 AI 동일 과제 묶음과 비교 전문:
kds0206/문서/260912_external_ai_task/ (05_compare.md)
- 9월 14일 AI 검토 결과와 발표 준비 점검서(핸드폰용 PDF):
kds0206/문서/260914_발표준비_점검서_4AI.pdf, 검토 원문 ~/.council/260914_발표준비판정/ 와 ~/.council/260914_교정/, 이 수정판 파일
kds0206/문서/260914_노선불여넣기_실험설계정리_HamNet_

해석 시 주의

- 파일럿0 v2 의 48/64 결과는 복숭아를 뺀 세 데이터셋의 센터크롭 중간 집계 결과다.
- 센터크롭 Dice 만으로 전체 프로토콜을 확정하지 않는다.
- 첫 파일럿0 의 runs 숫자는 인용하지 않는다.
- 모델·체크포인트·실험 결과의 정본은 학교 서버 platform 문서와 runs_v2 다.

2부. HamNet 모델 공부

HamNet 모델 공부 — Attention 대신 행렬분해로 전역 문맥 잡기 (2026-09-14 수정판)

작성일: 2026-09-13 / 최종 수정: 2026-09-14 발표자: 곽동신
용도: 첫 모델 공부 발표 및 12×12 백본·헤더 비교 실험 이해
논문: Geng et al., Is Attention Better Than Matrix Decomposition?, ICLR 2021

1. 핵심 요약

HamNet은 self-attention 대신 행렬분해, 특히 NMF를 이용해 특징맵 전체의 공통 패턴을 압축·복원하는 의미론적 분할 모델이다.

- 핵심 모듈 이름은 Hamburger 다.
- Hamburger는 선형변환(Lower bread)–행렬분해(Ham)–선형변환(Upper bread)의 빵–햄–빵 구조다.
- 모든 픽셀 쌍을 비교하지 않고 특징맵 전체를 소수의 대표 패턴으로 요약한다.
- 전역 self-attention 보다 계산량과 메모리 사용량을 줄이는 것이 핵심 목적이다.
- 우리 12개 헤더 중 행렬분해 문맥 계열을 대표하는 유일한 헤더다.
- 주의: 원 논문의 HamNet은 ResNet 백본에 Hamburger를 붙인 모델이다. LightHamHead는 같은 NMF Hamburger를 가벼운 분할 디코더로 쓰는 SegNeXt(NeurlIPS 2022) 계열 구현이다. 우리 표에서는 비교 편의를 위해 HamNet (LightHam)으로 묶어 적었다.

2. 왜 이 모델이 나왔는가

- 의미론적 분할에서는 멀리 떨어진 픽셀 사이의 관계도 봐야 한다. 이미지 양쪽에 떨어진 블루베리가 비슷한 색·모양·질감을 갖는다면 같은 과일 영역으로 인식할 수 있어야 한다.
- 전역 self-attention은 모든 위치 쌍의 관계를 계산해 전역 문맥을 얻는다. 위치가 n 개이면 대략 $n \times n$ 관계 행렬을

만들어야 하므로 고해상도 특징맵에서 메모리와 계산량이 크게 늘다.

- 논문의 질문: 모든 픽셀 쌍을 직접 비교하지 않고도 특징맵 전체의 공통 구조를 찾을 수 있는가?
- 논문의 답: 특징을 소수의 대표 패턴으로 분해했다가 다시 복원하는 행렬분해를 쓰자.

3. NMF 를 쉽게 이해하기

- 백본이 만든 특징맵을 ReLU 로 0 이상으로 만든 뒤 펼치면 $X \in \mathbb{R}^{(C \times HW)}$ 형태의 큰 행렬이 된다. C 는 채널 수, $H \times W$ 는 위치 수, X 의 한 열은 한 위치의 특징 벡터다.
- NMF(Non-negative Matrix Factorization, 비음수 행렬분해)는 이 큰 행렬을 두 개의 작은 행렬로 나눈다: $X \approx D \times C_code$. X, D, C_code 의 원소는 모두 0 이상이다. (우리 구현에서는 2D 특징맵용 NMF2D 모듈)
- D : 특징맵 전체에서 반복되는 대표 패턴을 모은 사전
- C_code : 각 위치가 대표 패턴을 얼마나 섞어 쓰는지 나타내는 계수
- $D \times C_code$: 저랭크로 다시 만든 특징. 중복과 잡음이 줄어들 것을 기대한다(보장은 아니다).
- 블루베리 비유: D 는 «푸른 과실 표면», «잎», «줄기», «반사광», «그림자» 같은 반복 패턴의 색인, C_code 는 각 픽셀의 배합표. 실제 학습에서 각 패턴이 사람이 이해할 수 있는 의미로 나뉜다는 보장은 없다. 계산 원리를 이해하기 위한 비유다.

4. Hamburger 구조

백본 특징 → Lower bread → NMF Ham → Upper bread
→ BN → 원래 특징과 더하기

- Lower bread: 입력 채널을 NMF 가 처리하기 좋은 특징 공간으로 바꾼다.
- Ham: NMF 로 대표 패턴 D 와 위치별 계수 C_code 를 반복 갱신한다.
- 저랭크 복원: $D \times C_code$ 로 특징맵 전체의 공통 구조를 반영한 특징을 만든다.
- Upper bread: 채널을 원래 특징 공간으로 되돌린다.

- Skip connection: 원래 입력을 더해 세부 정보를 보존한다.

논문 표현: $H(Z) = W_u M(W_l Z)$, 최종 출력 $Y = Z + BN(H(Z))$

5. 왜 전역 문맥을 잡을 수 있는가

- 대표 패턴 D 는 한 위치만 보고 만들어지지 않고 특징맵의 모든 위치를 함께 써서 갱신된다.
- 복원된 각 위치도 같은 대표 패턴을 공유한다. 따라서 픽셀마다 다른 모든 픽셀과 관계 행렬을 만들지 않아도 전체의 반복 구조를 전달받는다.

14일에 고친 곳

«14일 수정» 여기서 «전체»는 모듈에 들어온 특징맵 전체다. 우리 추론은 512×512 창을 밀어 가며 하므로 실제 범위는 «창 안 전체»이고, 멀리 떨어진 창끼리 정보를 나누는 것은 아니다.

6. Self-attention 과 차이

- 전역 정보 획득: (attention) 모든 위치 쌍의 유사도 계산 / (Hamburger NMF) 전체 특징을 대표 패턴으로 분해·복원
- 주요 중간 행렬: (attention) $n \times n$ attention map / (NMF) $d \times r, r \times n$ 저랭크 행렬
- 계산 복잡도: (attention) 대략 $O(n^2 d)$ / (NMF) 대략 $O(n d r) \times$ 반복 횟수, 보통 $r \ll n$
- 강점: (attention) 위치 간 관계를 직접적이고 유연하게 표현 / (NMF) 고해상도에서 비교적 가볍고 메모리 친화적
- 위험: (attention) 해상도가 커지면 비용 급증 / (NMF) rank·반복 횟수에 민감하고 경계가 평활화될 수 있음

14일에 고친 곳

«14일 수정» 여기서 attention 은 전역(모든 위치 쌍) self-attention 을 뜻한다. 창 attention(Swin 등)은 다르다.

7. One-step gradient 가 필요한 이유

- NMF 는 내부에서 여러 번 반복 계산한다. 모든 반복 단계를 그대로 역전파하면 기울기가 불안정해지고 반복 횟수에 따라 메모리도 는다.
- 논문은 앞선 반복에서는 gradient 를 끊고 마지막 한 단계만

미분하는 one-step gradient 를 썼다. 역전파의 시간·공간 복잡도가 반복 횟수 t 에 대한 $O(t)$ 에서 $O(1)$ 로 준다.

- 반복 자체는 그대로 한다. 끊는 것은 역전파뿐이다.
- NMF 와 CD(Concept Decomposition, 논문이 함께 시험한 다른 분해 방법) 는 약 6회 반복이면 충분했다. 너무 완전히 수렴시키면 단순한 행렬분해 가정에 과적합될 수 있어 3~6회가 적절하다고 보고했다.

8. 논문에서 기억할 결과

- Hamburger 전체: 78.3 best / 77.8 mean mIoU (PASCAL VOC ablation)
- 기본 모델: 75.9 / 75.7 mIoU (Hamburger 추가 전)
- Ham 제거: 75.8 / 75.6 mIoU → 핵심 이득이 행렬분해에서 발생
- Ham(NMF): 0.50M params / 17.6G MACs (논문의 동일 특징 입력 비교)
- Ham(NMF) 추론: 98MB / 7.7ms (TITAN Xp, $1 \times 512 \times 128 \times 128$ 입력) vs Self-attention 2148MB / 82.2ms
- HamNet: PASCAL VOC test 85.9 mIoU (COCO 사전학습 없이), PASCAL-Context val 55.2 mIoU (ResNet-101)
- 위 수치는 2021년 논문의 특정 장비·입력·학습 조건 값이다. 우리 실험이나 현재 GPU 수치로 그대로 쓰면 안 된다.

9. 우리 12×12 grid 에서의 역할

- 우리 헤더 구성은 CNN·pooling 7종, attention 3종, MLP 1종, matrix decomposition 1종이다.
HamNet/LightHam 은 matrix decomposition 계열을 대표하는 유일한 비교축이다.
- 확인할 질문: attention 없이도 전역 문맥을 충분히 잡을 수 있는가 / OCRNet 같은 attention 헤더와 비교했을 때 정확도·메모리·속도 균형은 어떤가 / 백본이 달라져도 LightHam 의 효율 이점이 유지되는가 / 작은 과실과 복잡한 경계에서 저랭크 압축이 도움이 되는가, 세부를 잃게 하는가

14일에 고친 곳

«14일 수정» 헤더끼리의 비교 결과는 «전역 문맥을 얻는 방식» 차이만이 아니라 채널 수·사용하는 특징 단계 같은 구현 차이도 함께 반영한다. 방식의 차이만 분리했다고 단정하지 않는다.

10. 블루베리 데이터에서 예상되는 장단점

기대되는 점

- 비슷한 과실 패턴이 화면 전체에 반복되므로 전역 대표 패턴을 만들기 좋을 수 있다.
- 고해상도 입력에서 $n \times n$ attention map 을 피하는 메모리 이점이 중요할 수 있다.
- 배경 잡음과 반사광을 줄이고 과실의 공통 특징을 강화할 가능성이 있다.
- 헤더가 가벼우면 많은 조합을 실험하거나 현장 장비에 배포할 때 유리하다.

불리할 수 있는 점

- 작은 과실 경계나 맞닿은 과실 사이의 가는 틈이 저랭크 복원에서 뭉개질 수 있다.
- 블루베리·그림자·잎이 비슷한 패턴으로 섞이면 false positive 가 늘 수 있다.
- 표본이 적은 복숭아에서는 대표 패턴이 안정적으로 형성되지 않을 수 있다.
- 원 논문은 PASCAL VOC·Context 중심이므로 밀집 과실에 유리하다는 직접 근거는 없다.

14일에 고친 곳

우리 구현(LightHam)은 백본의 가장 고해상도 단계(stage 0, 1/4)를 쓰지 않고 stage 1~3 만 쓴다. 작은 과실 경계에 불리할 수 있는 구체적 이유다. «14일 추가»

결론

«HamNet 은 밀집 과실에 유리하다»가 아니라 우리 실험에서 확인할 가설이다. 전체영상 Dice, 작은 객체 recall, 맞닿은 경계 오류, VRAM, 전체영상 처리시간을 함께 비교해야 한다.

11. 10분 발표용 다섯 칸 대본

1칸 — 한 줄 정의 "HamNet 은 모든 픽셀 쌍을 비교하는 self-attention 대신, 특징맵을 소수의 대표 패턴으로 분해했다가 복원하는 Hamburger 모듈을 쓰는 분할 모델입니다. 대표 행렬분해 방식으로 NMF 를 씁니다."

2칸 — 무엇을 해결했나 "분할에서는 멀리 떨어진 영역의 관계를 봐야 하지만 전역 self-attention 은 위치 수가 많아질수록 $n \times n$ 관계 행렬이 커집니다. HamNet 은 전체 특징을 저랭크 구조로 압축해 전역 문맥을 얻으면서 계산량과 메모리를 줄이려 했습니다."

3칸 — 구조 설명 "백본 특징이 아래쪽 빵을 지나 NMF 에 들어가고, NMF 는 대표 패턴과 위치별 계수로 나눕니다. 두 행렬을 다시 곱해 저랭크로 정리된 특징을 만들고, 위쪽 빵과 skip connection 을 거쳐 원래 세부 정보를 합칩니다."

4칸 — 우리 grid 에서의 자리 "우리 12개 헤더 중 HamNet/LightHam 은 행렬분해 문맥 계열을 대표하는 유일한 모델입니다. OCRNet 같은 attention 계열과 비교해 전역 문맥을 얻는 방식의 차이를 볼 수 있습니다. 우리 공용 코드의 LightHamHead 는 진짜 NMF 구현이고, rank 64 에 백본 stage 1~3 을 씁니다."

5칸 — 블루베리에서의 가설 "반복되는 과실 패턴과 큰 원본 영상을 다룰 때는 가볍고 전역 정보를 모으는 장점이 기대됩니다. 반대로 작은 과실 경계나 맞닿은 부분이 저랭크 복원으로, 그리고 1/4 해상도 단계를 쓰지 않는 구조 때문에 흐려질 수도 있습니다. 그래서 정확도뿐 아니라 작은 객체 recall, 경계 오류, VRAM 과 전체영상 처리시간을 함께 비교해야 합니다."

12. 예상 질문과 답변

- Q1. NMF 도 attention 인가? — 아니다. 둘 다 전역 문맥을 얻지만 attention 은 위치 간 유사도를 직접 계산하고, NMF 는 전체 특징을 저랭크 대표 패턴과 계수로 분해한다.
- Q2. NMF 의 N 은 무엇인가? — Non-negative. 분해 행렬의 값을 0 이상으로 제한한다. 논문은 NMF 에 넣기 전에 ReLU 를 적용한다.
- Q3. 왜 저랭크가 전역 정보를 뜻하는가? — 모든 위치를

소수의 공통 패턴으로 설명하기 때문이다. 각 위치가 전체에서 얻은 동일한 패턴 사전을 공유한다.

- Q4. 무조건 self-attention 보다 좋은가? — 아니다. 논문 조건에서는 효율적이고 경쟁력 있었지만 데이터와 구현에 따라 세부 경계 손실이나 rank 민감성이 생길 수 있다.
- Q5. HamNet 과 LightHam 은 같은가? — 원 논문의 HamNet 은 ResNet+Hamburger 모델이다. LightHamHead 는 NMF Hamburger 를 가벼운 분할 디코더로 쓰는 SegNeXt 계열 구현이다.
- Q6. 발표에 어떤 그림을 쓰면 좋은가? — 논문 Figure 1 의 빵-햄-빵 구조를 단순화한 흐름도. 결과를 설명할 때는 Figure 6 의 Hamburger 적용 전후 특징맵을 보조로.

14일에 고친 곳

Q7. 왜 stage 0 을 빼나? «14일 추가» — SegNeXt 원 구현을 그대로 따른 것이다. 그 영향(작은 경계)은 우리 실험의 확인 대상이다. 효율 때문이라고 단정하지 않는다.

13. 우리 구현 확인 상태 «14일 수정 — 확인 완료»

- 2026-09-14 서버에서 확인 완료. 공용 코드 platform/03_code/semantic-segmentation/semseg/model의 LightHamHead 는 진짜 NMF2D Hamburger 다(SegNeXt/mmseg ham_head 포팅).
- 설정: rank $R = 64$, ham_channels 256, NMF 반복 학습 6회 / 평가 7회, one-step gradient(앞 반복은 no_grad). 백본 stage 1~3($1/8 \cdot 1/16 \cdot 1/32$)만 사용, stage 0($1/4$) 제외. 래퍼 semseg/models/lightham.py, 두 __init__.py 에 등록.
- 개인 레포(kds0206/semantic-segmentation) 사본의 LightHamHead 는 NMF 가 없는 다른 구조(스케일별 scalar gate)다. 이 문서의 이전 판이 «오래된 코드로 판단»한 것이 맞았다. HamNet 결과는 반드시 공용 코드로 낸다.
- 12×12 grid 의 HamNet 슬롯 = 이 공용 구현. 동결기록(platform/planning/frozen/260908_체크포인트_동결 공용 03_code 사용을 명시한다.
- 논문과 «100% 동일»이라고까지는 말하지 않는다. mmseg 포팅이며 세부 설정(rank, 채널, 반복 수)은 SegNeXt

기본값이다.

14. 참고자료

- 논문: <https://arxiv.org/abs/2109.04553>
- ICLR 2021 논문 PDF:
<https://zhouchenlin.github.io/Publications/2021-ICLR-Attention>
- Hamburger 공식 구현:
<https://github.com/Gsunshine/Enjoy-Hamburger>
- SegNeXt 논문: <https://arxiv.org/abs/2209.08575>
- SegNeXt 공식 구현:
<https://github.com/Visual-Attention-Network/SegNeXt>

15. 발표 마지막 한 문장

HamNet은 attention을 더 복잡하게 만드는 대신, 오래된 행렬분해를 신경망 안에서 다시 해석해 가볍게 전역 문맥을 얻은 모델이며, 우리 연구에서는 그 효율성과 작은 과실 경계 보존 능력을 함께 검증해야 한다.